

第六次上机作业

学号：241830137 姓名：朱子健

2026 年 4 月 15 日

1 问题

求 $f(x) = e^x$ 在区间 $[0, 1]$ 上的 n 次最佳平方逼近多项式 $\varphi(x)$: $\varphi(x) = \sum_{k=0}^n a_k \varphi_k(x)$

- $n = 5$, $\varphi_k(x) = x^k$;
- $n = 5$, $\varphi_k(x)$ 为第 k 次 Legendre 多项式;
- $n = 10$, $\varphi_k(x) = x^k$;
- $n = 10$, $\varphi_k(x)$ 为第 k 次 Legendre 多项式.

2 算法原理与设计

2.1 算法原理

2.1.1 最佳平方逼近

设 $f(x) \in C[a, b]$, $\Phi_{n+1} \subset C[a, b]$ 为区间 $[a, b]$ 上的一个线性无关函数系 $\{\phi_i(x)\}_{i=0}^n$ 张成的空间。若存在 $\phi^*(x) \in \Phi_{n+1}$ 使得

$$\|f(x) - \phi^*(x)\|_2^2 = \min_{\phi(x) \in \Phi_{n+1}} \|f(x) - \phi(x)\|_2^2,$$

则称 $\phi^*(x)$ 为 $f(x)$ 在区间 $[a, b]$ 上的最佳平方逼近(函数)。这里 $\|\cdot\|_2^2$ 是 $C[a, b]$ 上带权 $W(x)$ 的内积, 即

$$\|f(x) - \phi(x)\|_2^2 = (f(x) - \phi(x), f(x) - \phi(x)) = \int_a^b [f(x) - \phi(x)]^2 W(x) dx.$$

设 $f(x) \in C[a, b]$, $\Phi_{n+1} = \text{span}\{\phi_0(x), \phi_1(x), \dots, \phi_n(x)\}$ 为给定的线性无关函数系。在 Φ_{n+1} 中寻求 $f(x)$ 的最佳平方逼近函数

$$\phi^*(x) = \sum_{j=0}^n c_j \phi_j(x),$$

即求系数 $c_0, c_1, \dots, c_n$ 使误差平方积分

$$\|f - \phi\|_2^2 = \int_a^b [f(x) - \sum_{j=0}^n c_j \phi_j(x)]^2 W(x) dx$$

达到最小。由极值必要条件 $\frac{\partial}{\partial c_i} = 0$ 导出法方程组（讲义式 (3.4.5)）：

$$\sum_{j=0}^n (\phi_j, \phi_i) c_j = (f, \phi_i), \quad i = 0, 1, \dots, n,$$

其中内积定义为

$$(\phi_j, \phi_i) = \int_a^b \phi_j(x) \phi_i(x) W(x) dx, \quad (f, \phi_i) = \int_a^b f(x) \phi_i(x) W(x) dx.$$

写成矩阵形式为

$$\begin{bmatrix} (\phi_0, \phi_0) & (\phi_1, \phi_0) & \cdots & (\phi_n, \phi_0) \\ (\phi_0, \phi_1) & (\phi_1, \phi_1) & \cdots & (\phi_n, \phi_1) \\ \vdots & \vdots & \ddots & \vdots \\ (\phi_0, \phi_n) & (\phi_1, \phi_n) & \cdots & (\phi_n, \phi_n) \end{bmatrix} \begin{bmatrix} c_0 \\ c_1 \\ \vdots \\ c_n \end{bmatrix} = \begin{bmatrix} (f, \phi_0) \\ (f, \phi_1) \\ \vdots \\ (f, \phi_n) \end{bmatrix}.$$

该方程组的系数矩阵为 Gram 矩阵，对称正定，当基函数线性无关时非奇异，故系数 c_j 存在唯一。

2.1.2 Legendre 多项式

当权函数 $W(x) = 1$ ，区间为 $[-1, 1]$ 时，可定义如下形式的正交多项式

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n} (x^2 - 1)^n, \quad n = 1, 2, \dots, \\ P_0(x) = 1.$$

我们称这样的正交多项式为 **Legendre 多项式**。Legendre 多项式满足三项递推关系：

$$(k+1)P_{k+1}(t) = (2k+1)tP_k(t) - kP_{k-1}(t), \quad k \geq 1.$$

此外，由 Rodrigues 公式可导出导数恒等式：

$$P'_{k+1}(t) - P'_{k-1}(t) = (2k+1)P_k(t). \quad (*)$$

对 I_k 应用一次分部积分：

$$\begin{aligned} I_k &= \int_{-1}^1 e^{ct} P_k(t) dt = \left[\frac{1}{c} e^{ct} P_k(t) \right]_{-1}^1 - \frac{1}{c} \int_{-1}^1 e^{ct} P'_k(t) dt \\ &= \frac{1}{c} (e^c - (-1)^k e^{-c}) - \frac{1}{c} \int_{-1}^1 e^{ct} P'_k(t) dt. \end{aligned} \quad (1)$$

利用恒等式 (*) 替换 $P'_k(t)$ ：

$$P'_k(t) = \frac{1}{2k+1} (P'_{k+1}(t) - P'_{k-1}(t)).$$

代入 (1) 中的积分项：

$$\int_{-1}^1 e^{ct} P'_k(t) dt = \frac{1}{2k+1} \left(\int_{-1}^1 e^{ct} P'_{k+1}(t) dt - \int_{-1}^1 e^{ct} P'_{k-1}(t) dt \right).$$

对右侧两积分再次分部积分（反向操作）：

$$\begin{aligned} \int_{-1}^1 e^{ct} P'_m(t) dt &= [e^{ct} P_m(t)]_{-1}^1 - c \int_{-1}^1 e^{ct} P_m(t) dt \\ &= (e^c - (-1)^m e^{-c}) - c I_m. \end{aligned}$$

将 $m = k + 1$ 与 $m = k - 1$ 代入，整理后得到关于 I_{k+1}, I_k, I_{k-1} 的递推式：

$$I_{k+1} = \frac{2k+1}{c} I_k - I_{k-1} - \frac{2k+1}{c} (e^c - (-1)^k e^{-c}). \quad (2)$$

代入 $c = 1/2$ ，即得数值递推公式：

$$I_{k+1} = 2(2k+1)I_k - I_{k-1} - 2(2k+1)(e^{1/2} - (-1)^k e^{-1/2}), \quad k = 1, 2, \dots$$

2.1.3 算法设计

算法 1 求函数 e^x 的最佳平方逼近多项式，基函数为 $\varphi_k(x) = x^k$

输入：逼近次数 n

输出：展开系数 $\{a_k\}_{k=0}^n$

- 1: **Step 1:** 计算右端向量 $\mathbf{b} = (b_0, b_1, \dots, b_n)^T$
 - 2: $b_0 \leftarrow e - 1$
 - 3: **for** $j = 1$ **to** n **do**
 - 4: $b_j \leftarrow e - j \cdot b_{j-1}$ {分部积分递推}
 - 5: **end for**
 - 6: **Step 2:** 构造法方程矩阵 $G \in \mathbb{R}^{(n+1) \times (n+1)}$
 - 7: **for** $i = 0$ **to** n **do**
 - 8: **for** $j = 0$ **to** n **do**
 - 9: $G_{ij} \leftarrow \frac{1}{i+j+1}$ {Hilbert 矩阵}
 - 10: **end for**
 - 11: **end for**
 - 12: **Step 3:** 解线性方程组 $G\mathbf{a} = \mathbf{b}$ ，得系数向量 $\mathbf{a}$
 - 13: **Step 4:** 输出 $\mathbf{a} = (a_0, a_1, \dots, a_n)^T$
-

算法 2 求函数 e^x 的最佳平方逼近多项式，基函数为 Legendre 多项式

输入：逼近次数 n

输出：展开系数 $\{a_k\}_{k=0}^n$

- 1: **Step 1:** 计算初值
 - 2: $I_0 \leftarrow 2(e^{0.5} - e^{-0.5})$
 - 3: $I_1 \leftarrow 6e^{0.5} + 2e^{-0.5}$
 - 4: **Step 2:** 正向递推 $I_2, I_3, \dots, I_n$
 - 5: **for** $k = 1$ **to** $n - 1$ **do**
 - 6: $I_{k+1} \leftarrow 2(2k+1)I_k - I_{k-1} - 2(2k+1)(e^{0.5} - (-1)^k e^{-0.5})$
 - 7: **end for**
 - 8: **Step 3:** 计算系数 a_k
 - 9: **for** $k = 0$ **to** n **do**
 - 10: $a_k \leftarrow \frac{2k+1}{2} \cdot e^{0.5} \cdot I_k$
 - 11: **end for**
 - 12: **Step 4:** 输出 $\{a_k\}_{k=0}^n$
-

3 结果分析

3.1 数值计算结果

利用分部积分递推分别求出幂基法方程右端项 $b_j = \int_0^1 e^x x^j dx$ 以及 Legendre 基积分 $I_k = \int_{-1}^1 e^{t/2} P_k(t) dt$, 进而得到两组最佳平方逼近多项式的展开系数。表 ?? 和表 ?? 分别列出 $n = 5$ 和 $n = 10$ 时的系数（保留 6 位有效数字）。

表 1: 幂基最佳平方逼近系数 a_k ($f(x) = e^x$ 在 $[0, 1]$ 上)

n	a_0	a_1	a_2	a_3	a_4	a_5
5	1.000027	0.998866	0.509920	0.140342	0.069395	-0.006249
10	1.000000	1.000000	0.500000	0.166667	0.041667	0.008333

n	a_6	a_7	a_8	a_9	a_{10}
10	0.001389	0.000198	0.000025	0.000003	0.000000

表 2: Legendre 基最佳平方逼近系数 c_k ($f(x) = e^x$ 在 $[0, 1]$ 上)

n	c_0	c_1	c_2	c_3	c_4	c_5
5	1.718282	0.845155	0.139864	0.013794	0.000930	0.000046
10	1.718282	0.845155	0.139864	0.013794	0.000930	0.000046

n	c_6	c_7	c_8	c_9	c_{10}
10	1.75e-6	5.3e-8	1.3e-9	2.7e-11	4.7e-13

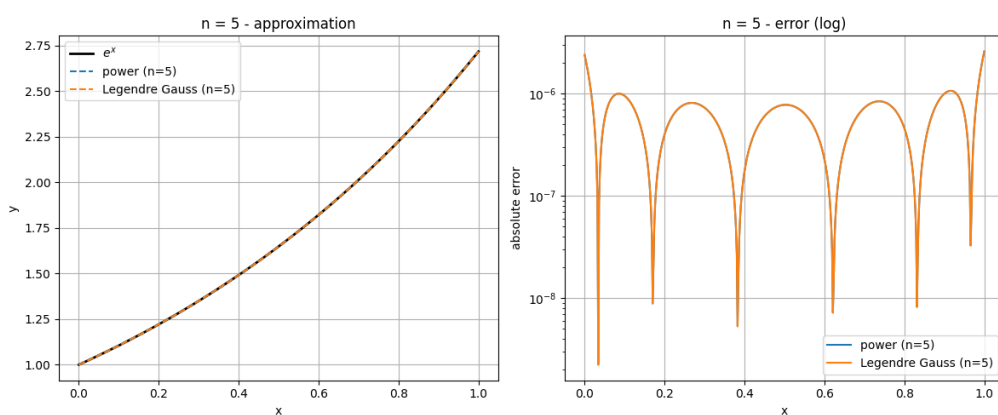


图 1: $n = 5$ 时的逼近曲线（左）与绝对误差（右）

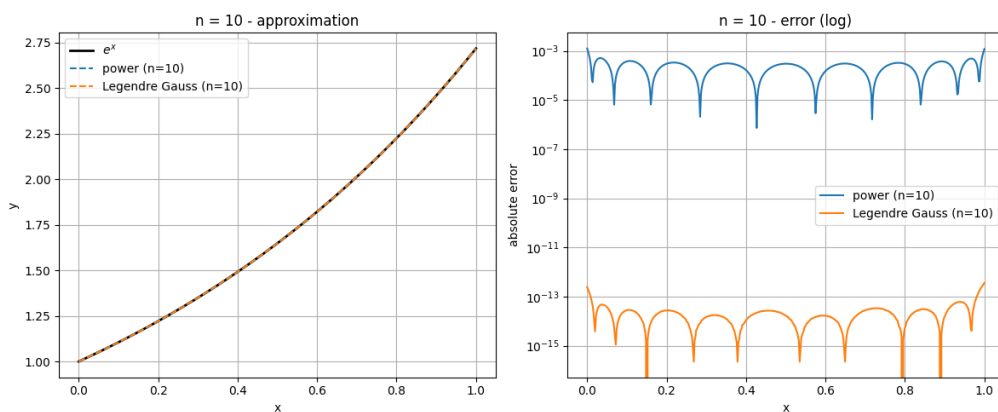


图 2: $n = 10$ 时的逼近曲线（左）与绝对误差（右）

3.2 数值计算结果分析

幂基对应的法方程矩阵为 Hilbert 矩阵 $\mathbf{H}_{n+1}$ ，其元素 $H_{ij} = 1/(i + j + 1)$ 。随着 n 增大，系数出现了偏差，这正是病态矩阵放大舍入误差的结果。Legendre 多项式在 $[-1, 1]$ 上关于权 $W(x) \equiv 1$ 正交，法方程矩阵为对角阵，无需解方程组，系数由内积直接计算。因此不存在病态问题，系数精度仅受积分计算精度控制。

4 附录: Python 源代码实现

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.special import eval_legendre
4 from numpy.polynomial.legendre import leggauss
5
6 def f(x):
7     return np.exp(x)
8
9 def power_basis_coeffs(n, high_precision=False):
10     if high_precision:
11         from decimal import Decimal, getcontext
12         getcontext().prec = 50
13         e = Decimal(np.e).exp()
14         b = [Decimal(0)] * (n + 1)
15         b[0] = e - Decimal(1)
16         for j in range(1, n + 1):
17             b[j] = e - Decimal(j) * b[j-1]
18         H = np.zeros((n + 1, n + 1), dtype=object)
19         for i in range(n + 1):
20             for j in range(n + 1):
21                 H[i, j] = Decimal(1) / Decimal(i + j + 1)
22         a = np.linalg.solve(H.astype(float), np.array(b, dtype=float))
23         return a
24     else:
25         b = np.zeros(n + 1)

```

```

26     b[0] = np.e - 1
27     for j in range(1, n + 1):
28         b[j] = np.e - j * b[j-1]
29     H = np.fromfunction(lambda i, j: 1.0 / (i + j + 1), (n + 1, n + 1))
30     a = np.linalg.solve(H, b)
31     return a
32
33 def power_approx(x, coeffs):
34     return np.polyval(coeffs[::-1], x)
35
36 def legendre_coeffs(n, method='gauss', N_gauss=50):
37     e_half = np.exp(0.5)
38     if method == 'gauss':
39         t, w = leggauss(N_gauss)
40         ft = e_half * np.exp(t / 2.0)
41         coeffs = np.zeros(n + 1)
42         for k in range(n + 1):
43             Pk = eval_legendre(k, t)
44             integral = np.sum(w * ft * Pk)
45             coeffs[k] = (2 * k + 1) / 2.0 * integral
46         return coeffs
47     elif method == 'recur':
48         e_n = np.exp(-0.5)
49         I = np.zeros(n + 1)
50         I[0] = 2.0 * (e_half - e_n)
51         if n >= 1:
52             I[1] = 6.0 * e_half + 2.0 * e_n
53         for k in range(1, n):
54             term = e_half - ((-1) ** k) * e_n
55             I[k+1] = 2.0 * (2 * k + 1) * I[k] - I[k-1] - 2.0 * (2 * k + 1) *
                    term
56         coeffs = (2.0 * np.arange(n + 1) + 1) / 2.0 * e_half * I
57         return coeffs
58
59 def legendre_approx(x, coeffs):
60     t = 2.0 * x - 1.0
61     n = len(coeffs) - 1
62     val = np.zeros_like(x)
63     for k in range(n + 1):
64         val += coeffs[k] * eval_legendre(k, t)
65     return val
66
67 def compute_errors(x_test, y_true, y_approx):
68     abs_err = np.abs(y_true - y_approx)
69     max_err = np.max(abs_err)
70     l2_err = np.sqrt(np.trapezoid((y_true - y_approx)**2, x_test))
71     return max_err, l2_err
72
73 def plot_results(x_plot, y_true, approx_dict, title):
74     plt.figure(figsize=(12, 5))

```

```

75 plt.subplot(1, 2, 1)
76 plt.plot(x_plot, y_true, 'k-', linewidth=2, label='$e^x$')
77 for label, y_app in approx_dict.items():
78     plt.plot(x_plot, y_app, '--', label=label)
79 plt.xlabel('x')
80 plt.ylabel('y')
81 plt.title(title + '_approximation')
82 plt.legend()
83 plt.grid(True)
84 plt.subplot(1, 2, 2)
85 for label, y_app in approx_dict.items():
86     err = np.abs(y_true - y_app)
87     plt.semilogy(x_plot, err, label=label)
88 plt.xlabel('x')
89 plt.ylabel('absolute_error')
90 plt.title(title + '_error_(log)')
91 plt.legend()
92 plt.grid(True)
93 plt.tight_layout()
94 plt.savefig(f'figure_n{title.split("=")[1].strip()}.pdf', bbox_inches='
    tight')
95 plt.show()
96
97 if __name__ == "__main__":
98     n_list = [5, 10]
99     x_plot = np.linspace(0, 1, 500)
100    y_true = f(x_plot)
101    for n in n_list:
102        print(f"\n{' '*60}\n\n_{n}\n{' '*60}")
103        a_pow = power_basis_coeffs(n, high_precision=False)
104        print(f"\npower_coeffs_(n={n}):")
105        for k, ak in enumerate(a_pow):
106            print(f"_{k}_={ak:15.10f}")
107        y_pow = power_approx(x_plot, a_pow)
108        max_err_pow, l2_err_pow = compute_errors(x_plot, y_true, y_pow)
109        print(f"max_error:_{max_err_pow:.4e},_L2_error:_{l2_err_pow:.4e}")
110        if n == 10:
111            a_pow_hp = power_basis_coeffs(n, high_precision=True)
112            print("\npower_coeffs_(high_precision,_n=10):")
113            for k, ak in enumerate(a_pow_hp):
114                print(f"_{k}_={ak:15.10f}")
115            c_leg_gauss = legendre_coeffs(n, method='gauss')
116            print(f"\nLegendre_coeffs_(Gauss,_n={n}):")
117            for k, ck in enumerate(c_leg_gauss):
118                print(f"_{k}_={ck:15.10f}")
119            y_leg_gauss = legendre_approx(x_plot, c_leg_gauss)
120            max_err_leg, l2_err_leg = compute_errors(x_plot, y_true, y_leg_gauss)
121            print(f"max_error:_{max_err_leg:.4e},_L2_error:_{l2_err_leg:.4e}")
122            c_leg_recur = legendre_coeffs(n, method='recur')
123            print(f"\nLegendre_coeffs_(recur,_n={n}):")

```

```

124     for k, ck in enumerate(c_leg_recur):
125         print(f"_{k}={ck:15.10f}")
126     y_leg_recur = legendre_approx(x_plot, c_leg_recur)
127     max_err_leg2, l2_err_leg2 = compute_errors(x_plot, y_true, y_leg_recur
128         )
129     print(f"max_error:{max_err_leg2:.4e},_L2_error:{l2_err_leg2:.4e}")
130     approx_dict = {
131         f'power_(n={n})': y_pow,
132         f'Legendre_Gauss_(n={n})': y_leg_gauss,
133     }
134     plot_results(x_plot, y_true, approx_dict, f'n={n}')
135     print("\n" + "="*60)
136     print("Hilbert_matrix_condition_numbers_(power_basis):")
137     for n in [5, 10]:
138         H = np.fromfunction(lambda i, j: 1.0/(i+j+1), (n+1, n+1))
139         print(f"n={n}:_cond(H)={np.linalg.cond(H):.2e}")
140     print("Legendre_basis_condition_number=1")

```